实验一

1

零函数调用开销：宏在预处理阶段展开，不产生函数调用，对于 GET_SIZE、HDRP 这类被高频调用的操作，性能提升明显。

代码可读性：GET_SIZE(HDRP(bp)) 比直接写 ((*unsigned int*)((char*)bp - 4) & ~0x07) 清晰得多。

便于修改：比如对齐方式从 8 改为 16，只需改 ALIGNMENT 宏，所有相关代码自动适配。

2

块的大小经过对齐后，低 3 位总是 0（因为大小是 8 的倍数）。因此可以利用最低位存储分配标志，而不影响大小本身。

3

脚部（ftr）的作用：每个块的末尾存储一份与头部相同的元信息。bp - DSIZE 正好是前一个块的脚部位置，从中可以读出前一个块的大小， 算出前一个块的起始地址：

PREV_BLKP(bp) = bp - GET_SIZE(bp - DSIZE)

对齐的作用：

保证所有块的起始地址和大小都是 8 的倍数，使头部/脚部低 3 位可用作标志。 满足硬件对 int、double 等类型的对齐访问要求。 简化 NEXT_BLKP 的计算：只需 bp + size。

代码

```
#include <stdio.h>

#include <stdlib.h>

#include <unistd.h>

#include <string.h>

/* ====================== 常量与对齐宏 ====================== */ #define ALIGNMENT 8

#define ALIGN(size) (((size) + (ALIGNMENT-1)) & ~(ALIGNMENT-1))

#define WSIZE 4

#define DSIZE 8

#define CHUNKSIZE (1<<12)
```

#define MINBLOCK (DSIZE + 2*WSIZE)

#define MAX(x, y) ((x) > (y) ? (x) : (y))

/* ========== 读写块字段的宏 ========== */ #define PACK(size, alloc) ((size) | (alloc))

#define GET(p) (*(unsigned int *)(p))

#define PUT(p, val) (*(unsigned int *)(p) = (val))

#define GET_SIZE(p) (GET(p) & ~0x07)

#define GET_ALLOC(p) (GET(p) & 0x1)

#define HDRP(bp) ((char*)(bp) - WSIZE)

#define FTRP(bp) ((char*)(bp) + GET_SIZE(HDRP(bp)) - DSIZE)

#define NEXT_BLKP(bp) ((char*)(bp) + GET_SIZE((char*)(bp) - WSIZE))

#define PREV_BLKP(bp) ((char*)(bp) - GET_SIZE((char*)(bp) - DSIZE))

/* ========== 全局状态 ========== */ static char *heap_listp;

#ifdef DEBUG

#define VERBOSE 1

#else

#define VERBOSE 0

#endif

/* ========== 前向声明 ========== */ static void *extend_heap(size_t size);

static void *find_fit(size_t size);

static void place(char *bp, size_t asize);

static void *coalesce(void *bp);

static void print_heap_blocks(int verbose, const char *func);

/**

- @brief 初始化内存分配器 */ int mm_init(void) { if ((heap_listp = sbrk(4WSIZE)) == (void*)-1) return -1;

  PUT(heap_listp, 0);

  PUT(heap_listp + 1*WSIZE, PACK(8, 1));

  PUT(heap_listp + 2*WSIZE, PACK(8, 1));

PUT(heap_listp, PACK(0, 1));

heap_listp += (2*WSIZE);

if (extend_heap(CHUNKSIZE) == NULL)

```
return -1;
```

return 0; }

/**

- @brief 打印当前堆块的状态信息 */ static void print_heap_blocks(int verbose, const char func) {

if (!verbose) return;

char *curbp = (char *)heap_listp;

printf("\n========== %s ==========\n", func);

while (GET_SIZE(HDRP(curbp)) > 0) {

```
printf("Address: %p\n", curbp);

printf("Size: %u\n", GET_SIZE(HDRP(curbp)));

printf("Allocated: %u\n\n", GET_ALLOC(HDRP(curbp)));

curbp = NEXT_BLKP(curbp);
```

}

printf("Address: %p\n", curbp);

printf("Size: 0\n");

printf("Allocated: 1\n");

printf("========== %s ==========\n", func); }

/**

- @brief 扩展堆的大小 */ static void *extend_heap(size_t size) {

char *bp;

size_t asize;

// 对齐到 8 字节，且不小于最小块大小 asize = ALIGN(size);

if (asize < MINBLOCK) asize = MINBLOCK;

// sbrk 返回旧 brk，即新区域的起始地址（也是新块的有效载荷起始位置） if ((bp = sbrk(asize)) == (void*)-1)

```
return NULL;
```

// 初始化新空闲块的头部和脚部 PUT(HDRP(bp), PACK(asize, 0));

PUT(FTRP(bp), PACK(asize, 0));

// 在新块末尾设置新的尾声头部 PUT(HDRP(NEXT_BLKP(bp)), PACK(0, 1));

// 尝试与前面的空闲块合并 return coalesce(bp); }

/**

- @brief 在堆中查找合适的空闲块（首次适配） */ static void *find_fit(size_t size) {

char *bp = heap_listp;

while (GET_SIZE(HDRP(bp)) > 0) {

```
if (!GET_ALLOC(HDRP(bp)) && GET_SIZE(HDRP(bp)) >= size)
    return bp;

bp = NEXT_BLKP(bp);
```

} return NULL; }

/**

- @brief 将内存块分配给用户 */ static void place(char *bp, size_t asize) { size_t csize = GET_SIZE(HDRP(bp));

size_t remainder = csize - asize;

if (remainder >= MINBLOCK) { // 分割：前 asize 字节分配出去，剩余部分作为新的空闲块

```
PUT(HDRP(bp), PACK(asize, 1));

PUT(FTRP(bp), PACK(asize, 1));

bp = NEXT_BLKP(bp);

PUT(HDRP(bp), PACK(remainder, 0));

PUT(FTRP(bp), PACK(remainder, 0));
```

} else { // 剩余太小，整块分配

```
PUT(HDRP(bp), PACK(csize, 1));

PUT(FTRP(bp), PACK(csize, 1));
```

} }

/**

- @brief 分配指定大小的内存块 */ void *mm_malloc(size_t size) { size_t asize;

  char *bp;

  if (size == 0) return NULL;

  // 加上头部和脚部，并对齐 if (size <= DSIZE) asize = 2 * DSIZE; else asize = ALIGN(size + DSIZE);

  // 先尝试在空闲块中查找 if ((bp = find_fit(asize)) != NULL) {

```
place(bp, asize);

print_heap_blocks(VERBOSE, "mm_malloc");

return bp;
```

  }

  // 没有合适的空闲块，扩展堆 if ((bp = extend_heap(MAX(asize, CHUNKSIZE))) == NULL) return NULL;

  place(bp, asize);

  print_heap_blocks(VERBOSE, "mm_malloc");

  return bp; }

/**

- @brief 合并相邻的空闲块 */ static void *coalesce(void *bp) {

  size_t prev_alloc = GET_ALLOC(FTRP(PREV_BLKP(bp)));

  size_t next_alloc = GET_ALLOC(HDRP(NEXT_BLKP(bp)));

  size_t size = GET_SIZE(HDRP(bp));

  if (prev_alloc && next_alloc) { // 前后都已分配，不合并 } else if (prev_alloc && !next_alloc) {

```
// 只有后一个块空闲
size += GET_SIZE(HDRP(NEXT_BLKP(bp)));
```

```
    PUT(HDRP(bp), PACK(size, 0));

    PUT(FTRP(bp), PACK(size, 0));
```

} else if (!prev_alloc && next_alloc) { // 只有前一个块空闲 size += GET_SIZE(HDRP(PREV_BLKP(bp)));

```
    PUT(FTRP(bp), PACK(size, 0));

    PUT(HDRP(PREV_BLKP(bp)), PACK(size, 0));

    bp = PREV_BLKP(bp);
```

} else { // 前后都空闲

```
    size += GET_SIZE(HDRP(PREV_BLKP(bp))) + GET_SIZE(HDRP(NEXT_BLKP(bp)));

    PUT(HDRP(PREV_BLKP(bp)), PACK(size, 0));

    PUT(FTRP(NEXT_BLKP(bp)), PACK(size, 0));

    bp = PREV_BLKP(bp);
```

}

return bp; }

/**

- @brief 释放一个已分配的内存块 */ void mm_free(void *ptr) {

  if (ptr == NULL) return;

  size_t size = GET_SIZE(HDRP(ptr));

  PUT(HDRP(ptr), PACK(size, 0));

  PUT(FTRP(ptr), PACK(size, 0));

  coalesce(ptr);

  print_heap_blocks(VERBOSE, "mm_free"); }

// 测试用例 int main() { if (mm_init() == -1) {

```
    printf("Memory manager initialization failed.\n");
    return -1;
}
```

```
  void *p1 = mm_malloc(8000);

  void *p2 = mm_malloc(200);

  void *p3 = mm_malloc(50);

  mm_free(p2);

  void *p4 = mm_malloc(150);

  mm_free(p1);

  mm_free(p3);

  mm_free(p4);

  return 0;
```

}

怎么处理内部碎片？

内部碎片主要来自两处：

对齐填充：请求 size 加上头脚后，ALIGN 到 8 的倍数，多出的部分就是内部碎片。例如请求 13 字节，13 + 8 = 21，ALIGN(21) = 24，浪费 3 字 节。

最小块限制：MINBLOCK = 16 字节，任何小于 16 的请求都会得到 16 字节的块。

这些是必要的开销，无法完全消除。但 place 中的分割机制避免了"大块被小请求占满"的浪费。

合并和分割空闲块会产生多余的 hdr 和 ftr 吗?

会。

分割：一个空闲块切成两个块，会多出一对头部和脚部（8 字节）。

合并：两个空闲块合并成一个，会减少一对头部和脚部（节省 8 字节）。

所以频繁分割和合并会增加元数据开销。实际分配器会通过延迟合并、分离链表等策略减少这种开销。

隐式分配器的时间复杂度是多少？为什么?

O(n)，n 是堆中块的总数。 因为 find_fit 需要从堆头开始，逐个块跳转，包括已分配的块，直到找到合适的空闲块或遍历完整个堆。隐式链表中没有单独维护空闲块列表，只能靠头 部大小信息逐个跳。

怎么提升性能?

显式空闲链表：只遍历空闲块，跳过已分配块。

分离空闲链表：按大小分类维护多个空闲链表，快速定位合适大小的块。

位图：用位图记录哪些块空闲，快速跳过连续已分配块。

首次适配优化：记录上次查找结束的位置，下次从那里继续。

mmap 大块：大请求直接用 mmap 分配，避免扩展堆。

ftr 的作用是什么？（深入理解后）

ftr 使得向前合并成为可能。释放一个块时，需要知道前一个块的大小和分配状态。没有 ftr，只能从堆头开始遍历到当前块才能找到前一个块，代价是 O (n)。有了 ftr，PREV_BLKP(bp) 可以在 O(1) 时间内定位前一个块。

"对外是数组，对内是链表"：物理上堆是连续内存，但通过块大小和指针计算，逻辑上形成了链表结构。用户看到的是独立的内存块，内部通过元信息隐式 链接所有块。这就是"隐式"的含义——没有显式指针，但通过头部信息隐式串起了所有块。

为什么不在每次搜索不到足够大的空闲块时进行遍历合并?

遍历合并需要扫描整个堆，时间复杂度 O(n)，如果每次 malloc 失败都做一次，性能会严重下降（吞吐量降低）。而且合并后未必能找到足够大的块，可 能还要扩展堆，做无用功。

相对可行的合并方法：

立即合并（Immediate Coalescing）：free 时立即检查并合并前后相邻空闲块。这是本实验的做法，每次 free 只需 O(1) 时间检查前后两块。

延迟合并（Deferred Coalescing）：free 时只标记空闲，不立即合并；等到 malloc 需要时再合并。可以减少合并次数，但可能产生更多碎片。

分离链表 + 分箱合并：按大小分类维护空闲链表，只合并同箱内相邻的块。

主流做法是 free 时立即合并，因为每次 free 的合并开销很小，且能及时控制碎片。

实验二

```
#include <stdio.h>

#include <stdlib.h>

#include <assert.h>

#include <unistd.h>
```

```
#include <string.h>

#define WSIZE 8

#define DSIZE 16

#define CHUNKSIZE (1 << 12)

#define MAX_HEAP (5 * (1 << 20))

#define MIN_BLK_SIZE (2 * DSIZE)

#define ALIGN(size) (((size) + (DSIZE - 1)) & ~(DSIZE - 1))

#define MAX(x, y) ((x) > (y) ? (x) : (y))

#define PACK(size, prev_alloc, alloc)
(((size) & ~0x7) | ((prev_alloc) << 1) | (alloc))

#define PACK_PREV_ALLOC(val, prev_alloc) (((val) & ~0x2) | ((prev_alloc) << 1))

#define PACK_ALLOC(val, alloc) (((val) & ~0x1) | (alloc))

#define GET(p) (*(unsigned long *)(p))

#define PUT(p, val) (*(unsigned long *)(p) = (val))

#define GET_SIZE(p) (GET(p) & ~0x7)

#define GET_ALLOC(p) (GET(p) & 0x1)

#define GET_PREV_ALLOC(p) ((GET(p) >> 1) & 0x1)

#define HDRP(bp) ((char *)(bp) - WSIZE)

#define FTRP(bp) ((char *)(bp) + GET_SIZE(HDRP(bp)) - DSIZE)

#define GET_PRED(bp) ((char *)GET(bp))

#define SET_PRED(bp, val) (PUT(bp, (unsigned long)(val)))

#define GET_SUCC(bp) ((char *)GET((char *)(bp) + WSIZE))

#define SET_SUCC(bp, val) (PUT((char *)(bp) + WSIZE, (unsigned long)(val)))

#define NEXT_BLKP(bp) ((char *)(bp) + GET_SIZE(((char *)(bp) - WSIZE)))

#define PREV_BLKP(bp) ((char *)(bp) - GET_SIZE(((char *)(bp) - DSIZE)))

static char *heap_listp;

static char *free_listp;

static char *mem_start_brk;

static char *mem_brk;
```

static char *mem_max_addr;

#ifdef DEBUG

#define VERBOSE 1

#else

#define VERBOSE 0

#endif

static void print_heap_blocks(int verbose, const char* func) {

```
if (verbose) {

    char *curbp = (char *)heap_listp;

    printf("\n========================== %s ==========================\n",
func);

    while (GET_SIZE(HDRP(curbp)) > 0) {

        printf("Address: %p\n", curbp);

        printf("Size: %lu\n", GET_SIZE(HDRP(curbp)));

        printf("Allocated: %lu\n\n", GET_ALLOC(HDRP(curbp)));

        curbp = NEXT_BLKP(curbp);
    }
    printf("Address: %p\n", curbp);

    printf("Size: 0\n");

    printf("Allocated: 1\n");

    printf("========================== %s ==========================\n", func);
}
```

}

static void *extend_heap(size_t size_bytes);

static void *find_fit_first(size_t asize);

static void *find_fit_best(size_t asize);

static void place(void* bp, size_t asize);

static void* coalesce(void* bp);

static void delete_from_free_list(void *bp);

static void add_to_free_list(void *bp);

void *mm_sbrk(int incr) {

```
  char *old_brk = mem_brk;

  if (incr < 0 || (mem_brk + incr) > mem_max_addr) {
      return (void*)-1;
  }

  mem_brk += incr;

  return old_brk;
```

}

int mm_init(void) { if (mem_start_brk == NULL) { if ((mem_start_brk = sbrk(MAX_HEAP)) == (void*)-1) return -1;

```
      mem_brk = mem_start_brk;

      mem_max_addr = mem_start_brk + MAX_HEAP;
  }

  free_listp = NULL;

  if ((heap_listp = mm_sbrk(4*WSIZE)) == (void*)-1)
      return -1;

  PUT(heap_listp, 0);

  PUT(heap_listp + WSIZE, PACK(DSIZE, 1, 1));

  PUT(heap_listp + 2*WSIZE, PACK(DSIZE, 1, 1));

  PUT(heap_listp + 3*WSIZE, PACK(0, 1, 1));

  heap_listp += 2*WSIZE;

  if (extend_heap(CHUNKSIZE) == NULL)
      return -1;

  return 0;
```

}

static void *extend_heap(size_t size_bytes) { char *bp;

```
  size_t size;

  size_t prev_alloc;


  size = ALIGN(size_bytes);

  if (size < MIN_BLK_SIZE) size = MIN_BLK_SIZE;

  if ((bp = mm_sbrk(size)) == (void*)-1) return NULL;


  prev_alloc = GET_PREV_ALLOC(HDRP(bp));

  PUT(HDRP(bp), PACK(size, prev_alloc, 0));

  PUT(FTRP(bp), PACK(size, prev_alloc, 0));

  PUT(HDRP(NEXT_BLKP(bp)), PACK(0, 0, 1));

  return coalesce(bp);
```

}

static void *find_fit_first(size_t asize) { char *bp = free_listp;

```
  while (bp != NULL) {
      if (GET_SIZE(HDRP(bp)) >= asize)

          return bp;

      bp = GET_SUCC(bp);
  }
  return NULL;
```

}

static void *find_fit_best(size_t asize) { char *bp = free_listp;

```
  char *best = NULL;

  size_t best_size = (size_t)-1;

  while (bp != NULL) {
      size_t sz = GET_SIZE(HDRP(bp));

      if (sz >= asize && sz < best_size) {
```

```
            best = bp;

            best_size = sz;

            if (sz == asize) break;
        }
        bp = GET_SUCC(bp);
    }
    return best;
```

}

static void place(void* bp, size_t asize) {

```
    size_t csize = GET_SIZE(HDRP(bp));

    size_t remainder = csize - asize;

    size_t prev_alloc = GET_PREV_ALLOC(HDRP(bp));

    delete_from_free_list(bp);

    if (remainder >= MIN_BLK_SIZE) {
        PUT(HDRP(bp), PACK(asize, prev_alloc, 1));

        char *new_bp = NEXT_BLKP(bp);

        PUT(HDRP(new_bp), PACK(remainder, 1, 0));

        PUT(FTRP(new_bp), PACK(remainder, 1, 0));

        add_to_free_list(new_bp);

        char *next = NEXT_BLKP(new_bp);

        PUT(HDRP(next), PACK_PREV_ALLOC(GET(HDRP(next)), 0));

    } else {
        PUT(HDRP(bp), PACK(csize, prev_alloc, 1));

        char *next = NEXT_BLKP(bp);

        PUT(HDRP(next), PACK_PREV_ALLOC(GET(HDRP(next)), 1));
    }
```

}

static void* coalesce(void* bp) { size_t prev_alloc = GET_PREV_ALLOC(HDRP(bp));

```
    size_t next_alloc = GET_ALLOC(HDRP(NEXT_BLKP(bp)));

    size_t size = GET_SIZE(HDRP(bp));

    size_t final_prev_alloc;

    if (prev_alloc && next_alloc) {
        /* 都不合并 */
    } else if (prev_alloc && !next_alloc) {
        size += GET_SIZE(HDRP(NEXT_BLKP(bp)));

        delete_from_free_list(NEXT_BLKP(bp));

    } else if (!prev_alloc && next_alloc) {
        size += GET_SIZE(HDRP(PREV_BLKP(bp)));

        delete_from_free_list(PREV_BLKP(bp));

        bp = PREV_BLKP(bp);
    } else {
        size += GET_SIZE(HDRP(PREV_BLKP(bp))) + GET_SIZE(HDRP(NEXT_BLKP(bp)));

        delete_from_free_list(PREV_BLKP(bp));

        delete_from_free_list(NEXT_BLKP(bp));

        bp = PREV_BLKP(bp);
    }

    final_prev_alloc = GET_PREV_ALLOC(HDRP(bp));

    PUT(HDRP(bp), PACK(size, final_prev_alloc, 0));

    PUT(FTRP(bp), PACK(size, final_prev_alloc, 0));

    char *next = NEXT_BLKP(bp);

    PUT(HDRP(next), PACK_PREV_ALLOC(GET(HDRP(next)), 0));

    add_to_free_list(bp);

    return bp;
```

}

static void delete_from_free_list(void *bp) { char *pred = GET_PRED(bp);

```
    char *succ = GET_SUCC(bp);

    if (pred != NULL) {
```

```
    SET_SUCC(pred, succ);

} else {
    free_listp = succ;
}
if (succ != NULL) {
    SET_PRED(succ, pred);
}
```

}

static void add_to_free_list(void *bp) { SET_PRED(bp, NULL);

```
SET_SUCC(bp, free_listp);

if (free_listp != NULL) {

    SET_PRED(free_listp, bp);

}
free_listp = bp;
```

}

void *mm_malloc(size_t size) { size_t asize;

```
char *bp;

if (size == 0) return NULL;

asize = ALIGN(size + WSIZE);

if (asize < MIN_BLK_SIZE) asize = MIN_BLK_SIZE;

if ((bp = find_fit_first(asize)) != NULL) {

    place(bp, asize);

    print_heap_blocks(VERBOSE, "mm_malloc");

    return bp;
}

size_t extendsize = MAX(asize, CHUNKSIZE);

if ((bp = extend_heap(extendsize)) == NULL)
    return NULL;

place(bp, asize);
```

```
  print_heap_blocks(VERBOSE, "mm_malloc");

  return bp;
```

}

void mm_free(void *bp) { if (bp == NULL) return;

```
  size_t size = GET_SIZE(HDRP(bp));

  size_t prev_alloc = GET_PREV_ALLOC(HDRP(bp));


  PUT(HDRP(bp), PACK(size, prev_alloc, 0));

  PUT(FTRP(bp), PACK(size, prev_alloc, 0));

  char *next = NEXT_BLKP(bp);

  PUT(HDRP(next), PACK_PREV_ALLOC(GET(HDRP(next)), 0));

  coalesce(bp);

  print_heap_blocks(VERBOSE, "mm_free");
```

}

void *mm_realloc(void *ptr, size_t size) { if (ptr == NULL) return mm_malloc(size);

```
  if (size == 0) { mm_free(ptr); return NULL; }

  size_t oldsize = GET_SIZE(HDRP(ptr));

  size_t asize = ALIGN(size + WSIZE);

  if (asize < MIN_BLK_SIZE) asize = MIN_BLK_SIZE;

  if (asize <= oldsize) return ptr;

  char *next = NEXT_BLKP(ptr);

  if (!GET_ALLOC(HDRP(next))) {

      size_t next_size = GET_SIZE(HDRP(next));

      if (oldsize + next_size >= asize) {

          delete_from_free_list(next);
```

```
            size_t newsize = oldsize + next_size;

            if (newsize - asize >= MIN_BLK_SIZE) {
                PUT(HDRP(ptr), PACK(asize, GET_PREV_ALLOC(HDRP(ptr)), 1));

                char *new_free = NEXT_BLKP(ptr);

                PUT(HDRP(new_free), PACK(newsize - asize, 1, 0));

                PUT(FTRP(new_free), PACK(newsize - asize, 1, 0));

                add_to_free_list(new_free);

                char *after = NEXT_BLKP(new_free);

                PUT(HDRP(after), PACK_PREV_ALLOC(GET(HDRP(after)), 0));

            } else {
                PUT(HDRP(ptr), PACK(newsize, GET_PREV_ALLOC(HDRP(ptr)), 1));

                char *after = NEXT_BLKP(ptr);

                PUT(HDRP(after), PACK_PREV_ALLOC(GET(HDRP(after)), 1));

            }
            print_heap_blocks(VERBOSE, "mm_realloc");

            return ptr;
        }
    }

    void *newptr = mm_malloc(size);

    if (newptr == NULL) return NULL;

    memcpy(newptr, ptr, oldsize - WSIZE);

    mm_free(ptr);

    return newptr;
```

}

int main() { if (mm_init() == -1) { printf("Memory manager initialization failed.\n");

```
        return -1;
    }

    void *p1 = mm_malloc(8000);

    void *p2 = mm_malloc(200);
```

```
    void *p3 = mm_malloc(50);

    mm_free(p2);

    void *p4 = mm_malloc(150);

    mm_free(p1);

    mm_free(p3);

    mm_free(p4);

    return 0;
```

}